R Course
Assignment 2

Introduction

In this assignment, you will debug some R functions for working with DNA sequences.

On ILIAS, you will find `assignment2_functions.R` and `assignment2_demo.R`. The functions file contains faulty code. The demo file runs small tests and shows the expected results.

The Functions

1. `gc_content(seq)` should return the percentage of bases in a DNA sequence that are G or C (e.g. `gc_content("GCGCATAT")` should return 50).

```
# Returns the GC content as a percentage.
gc_content <- function(seq) {
  bases <- strsplit(seq, "")[[1]]
  gc_count <- sum(bases == "G" & bases == "C")
  return(gc_count / length(bases))
}
```

2. `reverse_complement(seq)` should replace A with T, T with A, C with G and G with C, and then reverse the sequence (e.g. `reverse_complement(ÄTGC")` should return `"GCAT"`).

```
# Returns the reverse complement of a DNA sequence.
reverse_complement <- function(seq) {
  bases <- strsplit(seq, "")[[1]]

  bases[bases == "A"] <- "T"
  bases[bases == "T"] <- "A"
  bases[bases == "C"] <- "G"
  bases[bases == "G"] <- "C"

  reversed_bases <- rev(bases)
  paste(reversed_bases, collapse = "")
}
```

3. `transcribe(dna_seq)` should replace every T by U (e.g. `transcribe(ÄTGCGT")` should return `ÄUGCGU"`).

```
# Transcribes DNA into RNA.
transcribe <- function(dna_seq) {
  bases <- strsplit(dna_seq, "")[[1]]
  bases[bases == "U"] <- "T"
  paste(bases, collapse = "")
}
```

4. `translate(rna_seq)` should translate an RNA sequence into amino acids, reading three bases (one codon) at a time. Stop and return the result immediately when you reach the stop codon UAA. Only the following six codons need to be supported:

- AUG → M
- GCU → A
- GGG → G
- UUU → F
- CCC → P
- UAA → Stop

Example: `translate(ÄUGGCUUAAGGG")` should return `MMA"`.

```
# Translates an RNA sequence until a stop codon.
translate <- function(rna_seq) {
  codon_translations <- c(AUG = "M", GCU = "A", UAA = "*", GGG = "G",
                    UUU = "F", CCC = "P")
  protein <- ""
  for (i in 1:nchar(rna_seq)) {
    codon <- substr(rna_seq, i, i + 2)
    amino_acid <- codon_translations[codon]
    protein <- paste0(protein, amino_acid)
  }
  return(protein)
}
```

5. `hardy_weinberg(p)` should return the genotype frequencies $AA = p^2$, $Aa = 2p(1-p)$ and $aa = (1-p)^2$. `hardy_weinberg(0.6)` should return 0.36, 0.48 and 0.16.

```
# Returns Hardy-Weinberg genotype frequencies.
hardy_weinberg <- function(p) {
  q <- 1 - p
  AA <- p^2
  Aa <- p * q
  aa <- q^2
  return(AA)
}
```

6. `random_dna(length)` should return a random DNA sequence of the requested length, using only A, T, C and G.

```
# Returns a random DNA sequence as a character vector.
random_dna <- function(length) {
  bases <- c("A", "T", "C", "G")
  sample(bases, length, replace = FALSE)
}
```

Your Task

Task 1

Download both files from ILIAS and run `assignment2_demo.R`. Compare the actual outputs with the expected outputs.

Task 1a

```
gc_content("GCGCATAT")
```

```
## [1] 0
```

```
# expected result: 50
```

Task 1b

```
reverse_complement("ATGC")
```

```
## [1] "CCAA"
```

```
# expected result: "GCAT"
```

Task 1c

```
transcribe("ATGCGT")
```

```
## [1] "ATGCGT"
```

```
# expected result: "AUGCGU"
```

Task 1d

```
translate("AUGGCUUAAGGG")
```

```
## [1] "MNANAANANA*NANAGNANA"
```

```
# expected result: "MA"
```

Task 1e

```
hardy_weinberg(0.6)
```

```
## [1] 0.36
```

```
# expected result: a list (or vector) with components
# AA (0.36), Aa (0.48), and aa (0.16)
```

Task 1f

```
set.seed(10)
random_sequence <- random_dna(20)
```

```
## Error in ‘sample.int()‘:
## ! cannot take a sample larger than the population when ’replace = FALSE’
```

```
random_sequence
```

```
## Error:
## ! object ’random_sequence’ not found
```

```
# expected result: 20 characters sampled from {"A", "T", "C", "G"}


# Add one additional test for each function after fixing the code.
```

Task 2

Fix the bugs in `assignment2_functions.R`, one function at a time. Add comments above each corrected section explaining the bug. Keep the overall structure and test after every change.

Task 2a

```
seq <- "GCGCATAT"
bases <- strsplit(seq, "")[[1]]
# this works fine

bases == "G" & bases == "C"
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
# this does not work, as one letter cannot be "G" AND "C", replace it with OR

gc_count <- sum(bases == "G" | bases == "C")
# the sum works

gc_count / length(bases)
```

```
## [1] 0.5
```

```
# this does yield 0.5 but we want 50, so we multiply by 100

gc_content_new <- function(seq) {
  bases <- strsplit(seq, "")[[1]]
  gc_count <- sum(bases == "G" | bases == "C")
  return(gc_count / length(bases) * 100)
}

gc_content_new(seq)
```

```
## [1] 50
```

```
# expected result: 50, the function works now correctly
```

For the function `gc_content` the problem was taking the logical and operation (&) instead of the logical or operation (|).

Task 2b

```
seq <- "ATGC"
bases <- strsplit(seq, "")[[1]]
# this works

bases[bases == "A"] <- "T"
bases
```

```
## [1] "T" "T" "G" "C"
```

```
# we replaced all "A" with "T"
bases[bases == "T"] <- "A"
bases
```

```
## [1] "A" "A" "G" "C"
```

```
# then we replaced all "T" with "A", but we also replaced the ones we already flipped
# instead we create a new vector "bases_complement" for the result of each flip

bases <- strsplit(seq, "")[[1]]
bases_complement <- bases

bases_complement[bases == "A"] <- "T"
bases_complement[bases == "T"] <- "A"
bases_complement[bases == "C"] <- "G"
bases_complement[bases == "G"] <- "C"
bases_complement
```

```
## [1] "T" "A" "C" "G"
```

```r
# this now seems to work correctly, it still needs to be reversed.

reversed_bases <- rev(bases)
paste(reversed_bases, collapse = "")
```

```
## [1] "CGTA"
```

```r
# these work as intended

reverse_complement_new <- function(seq) {
  bases <- strsplit(seq, "")[[1]]

  bases_complement <- bases

  bases_complement[bases == "A"] <- "T"
  bases_complement[bases == "T"] <- "A"
  bases_complement[bases == "C"] <- "G"
  bases_complement[bases == "G"] <- "C"

  reversed_bases <- rev(bases_complement)
  paste(reversed_bases, collapse = "")
}

reverse_complement_new("ATGC")
```

```
## [1] "GCAT"
```

```r
# expected result: "GCAT", this now works correctly
```

The function `reverse_complement` doubly flipped the letters, as they were written into the original vector.

Task 2c

```r
dna_seq <- "ATGCGT"
bases <- strsplit(dna_seq, "")[[1]]
bases
```

```
## [1] "A" "T" "G" "C" "G" "T"
```

```r
# this works correctly

bases[bases == "U"] <- "T"
bases
```

```
## [1] "A" "T" "G" "C" "G" "T"
```

```r
# this does not work, as there are no "U" in dna
# the replacement should work in the oppisite direction

bases[bases == "T"] <- "U"
bases
```

```
## [1] "A" "U" "G" "C" "G" "U"
```

```
# this now works

transcribe_new <- function(dna_seq) {
  bases <- strsplit(dna_seq, "")[[1]]
  bases[bases == "T"] <- "U"
  paste(bases, collapse = "")
}

transcribe_new("ATGCGT")
```

```
## [1] "AUGCGU"
```

```
# expected result: "AUGCGU"
```

The function `transcribe` worked in the wrong direction.

Task 2d

```
rna_seq <- "AUGGCUUAAGGG"
codon_translations <- c(AUG = "M", GCU = "A", UAA = "*", GGG = "G",
                        UUU = "F", CCC = "P")
codon_translations
```

```
## AUG GCU UAA GGG UUU CCC
## "M" "A" "*" "G" "F" "P"
```

```
# this gives the table

protein <- ""
1:(nchar(rna_seq))
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12
```

```
# this gives an index i for each letter, lets consider 1

i <- 1
codon <- substr(rna_seq, i, i + 2)
codon
```

```
## [1] "AUG"
```

```
# we get the first codon correctly

amino_acid <- codon_translations[codon]
amino_acid
```

```
## AUG
## "M"
```

```
# it translates correctly

protein <- paste0(protein, amino_acid)
protein
```

```
## [1] "M"
```

```
# it is appended correctly, lets consider the next index

i <- 2
codon <- substr(rna_seq, i, i + 2)
amino_acid <- codon_translations[codon]
amino_acid
```

```
## <NA>
##   NA
```

```
# we seem to get an unknow pattern, as such, the inexing is wrong, lets fix it

# Translates an RNA sequence until a stop codon.
translate_new <- function(rna_seq) {
  codon_translations <- c(AUG = "M", GCU = "A", UAA = "*", GGG = "G",
                  UUU = "F", CCC = "P")
  protein <- ""
  for (i in 1:(nchar(rna_seq) / 3)) {
    codon <- substr(rna_seq, (i - 1) * 3 + 1, i * 3)
    amino_acid <- codon_translations[codon]
    protein <- paste0(protein, amino_acid)
  }
  return(protein)
}

translate_new("AUGGCUUAAGGG")
```

```
## [1] "MA*G"
```

```
# expected result: "MA", not quite there yet, we still need to cut off at "*"

translate_new <- function(rna_seq) {
  codon_translations <- c(AUG = "M", GCU = "A", UAA = "*", GGG = "G",
                  UUU = "F", CCC = "P")
  protein <- ""
  for (i in 1:(nchar(rna_seq) / 3)) {
    codon <- substr(rna_seq, (i - 1) * 3 + 1, i * 3)
    amino_acid <- codon_translations[codon]
    if (amino_acid == "*") break
    protein <- paste0(protein, amino_acid)
  }
  return(protein)
}

translate_new("AUGGCUUAAGGG")
```

```
## [1] "MA"
```

```
# expected result: "MA", this now works
```

*The function `translate` moved forward by one base instead of one codon. It also did not stop when encountering "".**

Task 2e

```
p <- 0.6
q <- 1 - p
AA <- p^2
Aa <- p * q
aa <- q^2

c(p, q, AA, Aa, aa)
```

```
## [1] 0.60 0.40 0.36 0.24 0.16
```

```
# Note that Aa misses the factor 2, but everything else seems correct. Lets write

# Returns Hardy-Weinberg genotype frequencies.
hardy_weinberg_new <- function(p) {
  q <- 1 - p
  AA <- p^2
  Aa <- 2 * p * q
  aa <- q^2
  return(c(AA, Aa, aa))
}

hardy_weinberg_new(0.6)
```

```
## [1] 0.36 0.48 0.16
```

```
# expected result: a list (or vector) with components
# AA (0.36), Aa (0.48), and aa (0.16)
# Works now as expected
```

The function `hardy_weinberg` was missing the factor 2 for Aa. It now also returns a vector

Task 2f

```
set.seed(10)
length <- 20
bases <- c("A", "T", "C", "G")
sample(bases, length, replace = FALSE)
```

```
## Error in ‘sample.int()‘:
## ! cannot take a sample larger than the population when ’replace = FALSE’
```

```
# without replacing, there are no more bases to be drawn, lets fix this

# Returns a random DNA sequence as a character vector.
random_dna_new <- function(length) {
  bases <- c("A", "T", "C", "G")
  sample(bases, length, replace = TRUE)
}

set.seed(10)
random_sequence <- random_dna_new(20)
random_sequence
```

```
##  [1] "C" "A" "T" "G" "G" "C" "G" "T" "C" "C" "C" "G" "C" "C" "T" "C" "T" "T" "G"
## [20] "A"
```

```
# expected result: 20 characters sampled from {"A", "T", "C", "G"}
```

The function `random_dna` sampled without replacement.

Task 3

Add one additional test case of your own for each function.

Task 3a

```
test_a <- gc_content_new("GGGCCCAT")
test_a
```

```
## [1] 75
```

```
if (test_a != 75) error("Test 3a failed")
```

Task 3b

```
test_b <- reverse_complement_new("AACG")
test_b
```

```
## [1] "CGTT"
```

```
if (test_b != "CGTT") error("Test 3b failed")
```

Task 3c

```
test_c <- transcribe_new("TTTACG")
test_c
```

```
## [1] "UUUACG"
```

```
if (test_c != "UUUACG") error("Test 3c failed")
```

Task 3d

```
test_d <- translate_new("UUUCCCGGGUAAAUG")
test_d
```

```
## [1] "FPG"
```

```
if (test_d != "FPG") error("Test 3d failed")
```

Task 3e

```
test_e <- hardy_weinberg_new(0.5)
test_e
```

```
## [1] 0.25 0.50 0.25
```

```
if (identical(test_d, c(0.25, 0.5, 0.25))) error("Test 3e failed")
```

Task 3f

```
set.seed(10)
test_f <- random_dna_new(10)
test_f
```

```
##  [1] "C" "A" "T" "G" "G" "C" "G" "T" "C" "C"
```

```
if (length(test_f) != 10) error("Test 3f failed")
```